

Manual

MES Development Suite Label Designer MDS-ETD 8.1

Version 1.1.23049

Last changed on: 02.09.2020

Copyright

©Copyright 2020 All rights reserved.

SAP® and R/3® are registered trademarks of SAP AG.

WINDOWS® is a registered trademark of Microsoft Corporation.

MPDV® and HYDRA® are registered trademarks of MPDV Mikrolab GmbH.

ORACLE® is a registered trademark of ORACLE Corporation, California, USA.

Copying and distribution of this documentation or any part thereof, for any purpose or in any form, is prohibited without prior written permission from MPDV Mikrolab GmbH.

The information contained in this documentation is subject to change without prior notice.

Contents

1	Overview – MES Development Suite Label Designer	4
2	Label Assignment.....	5
3	Label configuration	16
4	Schema Configuration.....	29
5	Scheme Detail Configuration	32
6	Label reprint	46
7	Label reprint via the terminal: configuration	49
8	List + Labels configurations	52
9	Central Configuration File hytnrcfg.ini	53
9.1	Layout configuration	56
10	Creation of data source DQDetail with HYDRA script	58
10.1	Creation of the HYDRA script.....	59
10.1.1	Troubleshooting.....	63
10.3	Introduction to the basic frame	64
10.3.1	Example 1: Simple list of persons.....	64
10.4	Introduction to extended options.....	66
10.4.1	Example 2: List of persons	66
10.5	Script examples for the HYDRA label design.....	69
10.5.1	Example 1: Basic frame script for person, machine	69

1 Overview – MES Development Suite Label Designer

Purpose

The Label Designer provides functions and tools for creating in-process documents such as time tickets, labels, material tickets, container identity cards, etc., which can be printed out in conjunction with the print interface for Windows terminals on printers connected to HYDRA shop floor terminals.

Implementation Notes

The function package is used where you wish to produce in-process documents.

Integration

The Label Designer allows the production and integration of your own documents. The respective print interface **AIP-AED** is required for the printout directly at the shop floor terminals in production.

On systems on which you only want to print your own documents that were created on another system, you can also use the runtime license **MDS-ETP** instead of the development license **MDS-ETD**.



For historical reasons, the **products** of the MES Development Suite differ from the **licenses** that must actually exist in the system:

Product (Price list)		License (in the system)
MDS-ETP	Runtime version label design	MDS-ETR

Features

The Label Designer

- Contains a form designer for creation of the forms at HYDRA clients
- Permits the creation of master-detail relationships for integration of local data on the shop floor terminal (master) and the optional data from the HYDRA database (detail).
- Offers the configuration possibility of different printed forms per data collection dialog.
- Offers the configuration possibility of different printed forms, depending on the current information from the respective data collection dialog (e.g. machine, article, material type, etc.).
- Permits printing of the master data when the shop floor terminal is offline.

2 Label Assignment

Overview

Menu	System administration => Label configuration => Label configuration
Transaction code	Iblass
Function authorization	Iblass

Label assignment

Main page Profile

Request data Cancel Print preview Print all Insert Copy Edit Delete Activate Deactivate Save Help on operation Help on application Help on detail application Help

Selection

Designation Active

Label

Label assignment

Drag a column here to group by that column

Configuration	Label type	Label	Suffix	Designation	Comment
> ca_wl_mpl	Label	ca_ab	qr3	Losetikett (...)	Losetikett (...)
a_un_mpl	Label	ca_ab	qr3	Losetikett (...)	Losetikett (...)
tth	Label	tth			
xxx1	Label	test			
xxx3	Label	x3			
a_tr	Label	a_tr	qr3	Test A_TR	
a_un_mpl	Label	ca_ab	qr3	Losetikett (...)	Losetikett (...)
x3	Label	x3			
a_tr	Label	ETD-Schulung		Training ETD	
a_tdm_jl2	Label	zebra2			
tthtest2	Label	tthmltest			
ETD-Training	Label	ETD-Training		MPDV	
TestCarina	Label	Carina		bla	
TestCarina	Label	Carina		bla	

Configuration

Assignment

Label

Suffix

Designation

Comment

☐ Active

☒ Protocol

☐ Print ☐ Print preview

☐ Batch print ☐ Data conversion

Quantity

Log. printer

Key value pairs

Key 1

Value 1

Key 2

Value 2

Assignment

Dialog

Processing

Last editing

Modified by

Modified on

Purpose

Labels can be assigned to the Windows data entry terminal. If an assignment is made and activated, the label is loaded from the HYDRA server when the Windows data collection terminal is started. The Windows collection terminal only loads the labels that are assigned to the selected Windows collection terminal.

Integration

Only existing labels can be assigned.

Requirements

Users must have the function authorization "lclass" in order to use this function. If users are authorized, they will have full access to all functions.

Function authorizations can also protect each editing application. These are described in more detail in the description of the editing application. If you have authorizations for an editing application, you are automatically authorized to access the main application where you may also view data.

Field descriptions

Assignment

Name of the label assignment.

Label

Name of the label from the label configuration.

Active

Indicator whether label assignment is active.

Log (former: Protocol)

If the checkbox is activated, the printing of the label is logged.

Information: The logging is required for the reprint function.

Quantity

Number of copies to be printed. Please note that you can override this configuration by the dynamic dialog field NUMPRN (see section: Prefix DLG).

Print mode

This attribute calculates if the label is printed or displayed in a print preview window.

Log. printer

A logical printer name can be defined here. This logical printer name is defined in the configuration of the print program and used when printing. If no value is defined, the standard printer is used.

Batch print

If this option is set only a batch program (BatchPrint.bat) is started instead of the print program.

Note:

The batch file is not in the standard. This means that the processing must be implemented via a customer request.

By means of the data conversion option, you can specify whether the data is transferred to the batch program in the dialog data format or in the dialog list format.

☒ → Dialog list format (MEMDATA - format)

☐ → Dialog data format

Assignment

Type

This option is used to control a label printout by a specific action on the terminal. The following values are supported:

No dialog assignment → The label is not printed automatically

Dialog assignment → The label is printed automatically after a message

Dialog assignment → The label is printed automatically after a message

Dialog

Dialog where this label should be printed.

Information: for all terminal dialogs that generate a database message, the printout is activated after a successful plausibility check of the dialog message (OK). Reading dialogs, e.g. "Shift Information" (M_SAW) do not trigger a label printing.

Processing

Final message or starting message.

Note:

A message completes a status and starts a new one. Machine status change => The time period for the current status is completed and the start time for the new status is started. The master data transferred by the terminal are identical in both cases. Only the script on the server can perform different actions based on the VERARB={A|B} value.

Key value pairs

The key value pairs are used to be able to print different labels at different terminals. It is also possible to print different labels at one workplace using one material type or one order type. **Note:** If the material type is chosen as assignment criterion, the field material type must be available in the dynamic dialog.

Example: Key-Value -Pairs Condition: Label printing for machine 00000100 / order type = 0 (a OP is currently logged with AART 0). Assignment 1 – 4 are active at the terminal 112.

Assignment	Dialog	Reference 1	Value 1	Reference 2	Value 2	Expression
Assignment 1	A_UN	Machine	00000100			No
Assignment 2	A_UN	Terminal	112			No
Assignment 3	A_UN	Machine	00000100	Order type	0	Yes
Assignment 4	A_UN	Terminal	112	Order type	1	No

The priority is decisive for calculating the assignment to be printed. Only assignments with the same priority are printed. If we need to have several labels printed, make sure that only labels with the same priority are processed.

The following table contains the rules used to identify a label:

Priority	Reference 1	Reference 2
1	Workplace**	CAQ - characteristics layout-
2	Workplace group**	CAQ - characteristics layout-
3	Terminal	CAQ - characteristics layout-
4	Terminal group	CAQ - characteristics layout-
5	Workplace	Operation layout*
6	Workplace group	Operation layout*
7	Terminal	Operation layout*
8	Terminal group	Operation layout*
9	Workplace	Material buffer***
10	Workplace group	Material buffer***
11	Terminal	Material buffer***
12	Terminal group	Material buffer***
13	Workplace	Material type layout*
14	Workplace group	Material type layout*
15	Terminal	Material type layout*
16	Terminal group	Material type layout*
17	Workplace	Material type
18	Workplace group	Material type
19	Terminal	Material type
20	Terminal group	Material type
21	Workplace	Order type****
22	Workplace group	Order type****
23	Terminal	Order type****
24	Terminal group	Order type****
25	(empty)	CAQ - characteristics layout-

Priority	Reference 1	Reference 2
26	(empty)	Operation layout*
27	(empty)	Material buffer***
28	(empty)	Material type layout*
29	(empty)	Material type
30	(empty)	Order type****
31	Workplace	(empty)
32	Workplace group	(empty)
33	Terminal	(empty)
34	Terminal group	(empty)

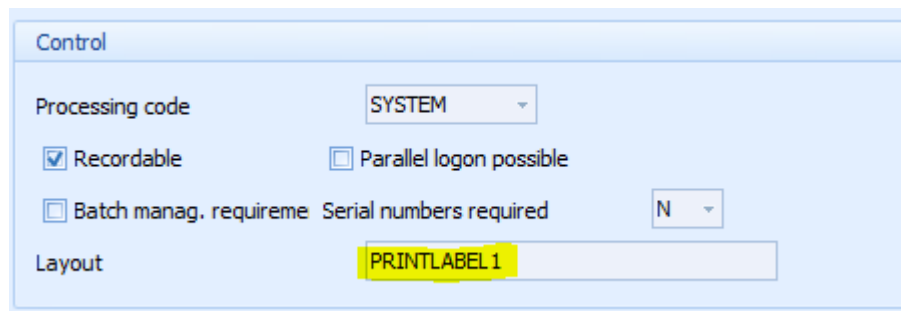
★ The relevant layout from the master data must be stored for these reference fields.

★★ Here, a value from the database must be stored in the value field. In case of numeric workplace numbers, the workplace or the workplace group has to be filled with leading zeroes.

★★★ *** The material buffer is primarily used from the respective dialog (destination field, ID ZLO). If there is no appropriate field in the dialog, the material buffer from the workplace list is used (field: subsequent material buffer).

★★★★ The order type is to be displayed in the relevant dialog. Here, the identifier AART from the Dynamic Dialog Configuration is used for identification.

Example: operation



Example: material type

The screenshot shows the 'General' tab of the configuration interface. It includes fields for 'Material type' (SYSTEM), 'Type' (E), 'Lot size' (0), 'Unit' (***), and 'Description' (System-Eintrag). Below these are tabs for 'Input batch processing', 'Output batch processing', 'General', and 'User fields'. The 'General' tab is active, showing 'Determination of expiry date' with radio buttons for 'Current point in time' (selected), 'Shift date', and 'Input batch'. Below this is the 'Ticket printing' section with 'Quantity' (1) and 'Layout' (LABELMPL1).



The key value pairs are not available for the label print in SMA.

Editor (Editor, date, time)

Edit / date / time of the last editing of the data record

Activated by (editor, date, time)

Editor / date / point in time when the user set the label active at last

Deactivated by (editor, date, time)

Editor / date / point in time when the user set the label inactive at last

Toolbar

In addition to the standard buttons of a configuration entry, there are two further buttons:

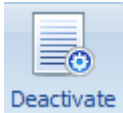


Activate

The label identified in the table can be activated with the aid of this button.

Note:

Only label assignments can be activated, if a report has been created for them. Label assignments, for which the "batch print" option is set, are excluded from this.



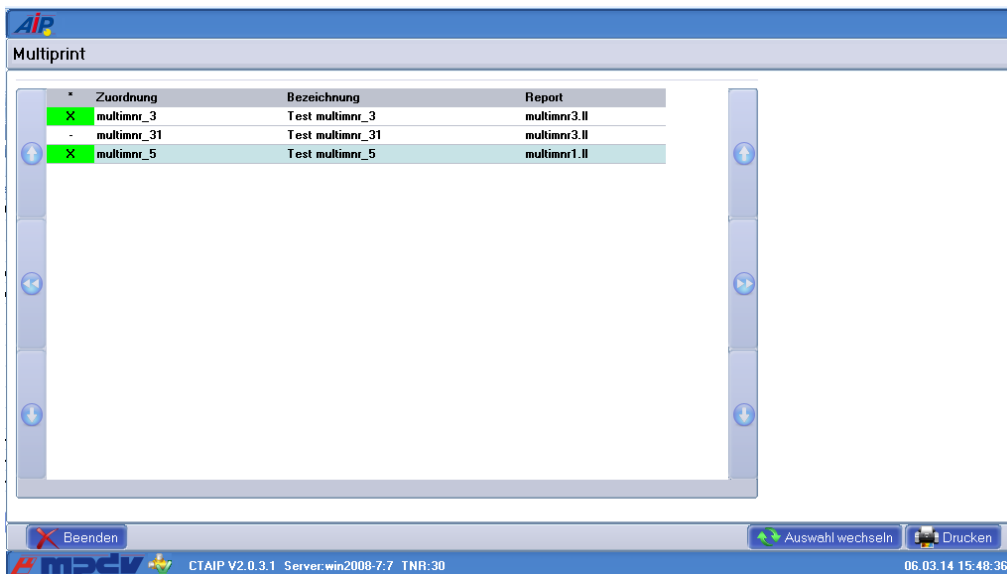
Deactivate

The label identified in the table can be deactivated with the aid of this button.

Multi print

It is possible from CTAIP version V# 2.0.3.4 to print different forms and labels at the same time on the terminal with one function.

In a selection dialog all labels / forms assigned to the multi-print dialog by the label assignment are displayed in a selection list for you.



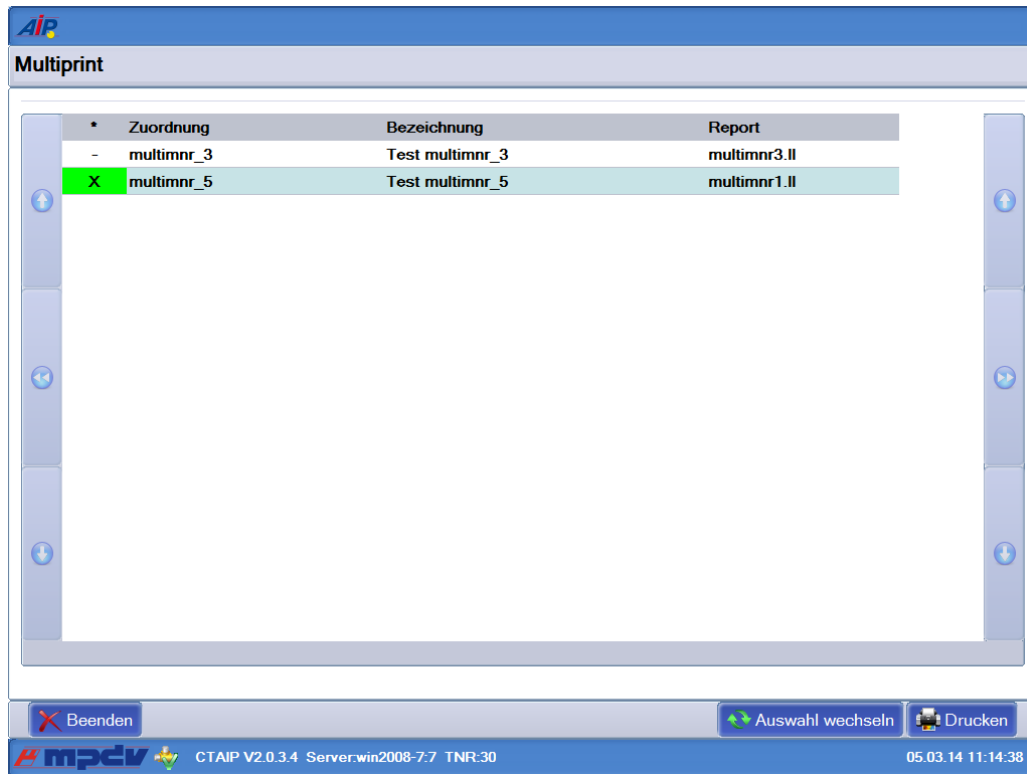
Note:

Do not activate more than 10-multi print labels on a terminal. The CT-AIP downloads from the server the multi-print labels during the program start.

The function "Exit" closes the selection dialog.

The function "Change selection" changes the selection of the entries.

The function "Print" prints all selected labels / forms.



All reports are listed in the selection dialog of the reports / forms that are assigned to the multi-print dialog.

Here, the configuration is included in the label assignment.

- „Key - values pairs“

This means only labels / forms assigned to the machine / terminal are displayed.

- Printer:

logic printer for the label / form. This means the multi-print dialog can issue the labels / forms on different printers.

That's why information is provided by the terminal for the labels.

There are different detail schemas to use

Schema / Detail schemas:

multidruck_mnr

=> all information is available for the selected machine

multidruck_anr

=> all information available for the selected machine

and the selected order

multidruck_cnr

=> all information available for the selected machine, selected order

and selected batch

Further data can be read according to standard label printing using detail scripts.

It is feasible to create different multi-print dialogs and to call them via different function buttons.

Configuration

- Reports / forms are designed as labels.
- The label assignment is done via the label assignment function for the multi-print catalog.

Note:

The selection dialog is not configurable but dynamically lists all assigned labels / forms

- All labels are collected when they are entered in the field "Dialog" of the multi-print dialog.
=> all labels with the same value in the field "Dialog" are shown in the multi-print window for the user to select.

- If necessary, the layout of the table can be changed in the file <ctaipplay.ini> in the MULTIPRINT section.

The default layout is as follows:

```
[MULTIPRINT]
GRID_FONT=Microsoft Sans Serif
GRID_FONTSIZE=9
GRID_COLOR=clBlack
GRID_BACKGROUND=clWhite
GRID_ORDER=ZUORDNUNG
GRID_CELLPAINT=ON
EXAMINE_CELLBKCOLOR=SELECT,SELECT,X-clLime
SELECT=C3,50,Z,*
ZUORDNUNG=C20,200,L,Zuordnung
CFGBEZ=C20,250,L,Bezeichnung
REPORT=C10,150,L,Report
```

- The multi-print dialog (selection dialog) is entered at the terminal into the configuration file for the terminal function button (ctaipbut.ini)

Tastennummer=MULTIPRINT.<Name selection dialog>,<Alignment>,<Text>,<Icon>

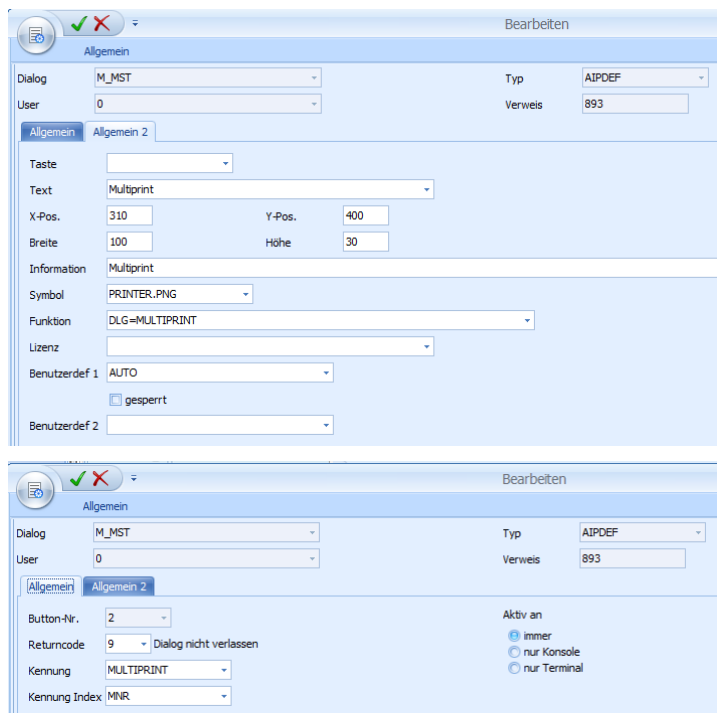
Example:

2=MULTIPRINT.MULTIPRINT_MNR,L,Multidruck,Printer.png

3=MULTIPRINT.MULTIPRINT_ANR,L,Multidruck Auftragsdaten,Printer.png

- Optional

The button for multi-print can also be called in a dialog using the configuration in a workflow step.



The configuration is done in the selection dialog of the fields „Acronym“ and „Acronym index“. These fields are restricted to 10 digits. If required, they are merged with a colon ":". (e.g. **MULTIPRINT:MNR**).

Patch for detail schema:

Execute the database patch *dbp_labelmultiprint.hsc* on the *HYDRA* server.

Windows:

```
hydscr.exe .\db_sql\dbp_labelmultiprint.hsc
```

Linux:

```
hydscr.out ./db_sql/dbp_labelmultiprint.hsc
```

3 Label configuration

Overview

Menu	System administration → Label configuration → Label configuration
Transaction code	lblcfg
Function authorization	lblcfg

Purpose

This application is used to manage and design labels in HYDRA.

Note on label printing on the terminal:

The labels support Unicode. All labels may be designed in any language. The master data are managed by the Windows terminal. The Windows terminal is not Unicode-compatible. The terminal manages the data in the local Windows code page. Consequently, only characters supported by the local Windows code page may be transferred to the printing program in these data. In optional detail data, data are provided by the HYDRA server in the Unicode format.

Requirements

In order that the user may apply this function, he/she must have the function authorization "lblcfg". If users are authorized, they will have full access to all functions.

Function authorizations can also protect each editing application. These are described in more detail in the description of the editing application. If you have authorizations for an editing application, you are automatically authorized to access the main application where you may also view data.

MOC

For label maintenance, the HYDRA path MOCREP is required. The path always has to refer to <MDT>/custom/reports. It is not allowed to change this.

Example configuration Windows (system 1):

Activation on the terminal

The "list & label" print must be activated. You need to configure the following entry in each INI file:

```
[Tnr Konfiguration 0]
QRD-PRINTER->REPORTTYPE=ll
```

The value "ll" is the abbreviation for "list & label": two lowercase letters "L".

INI file depending on the terminal :

AIP 2	<HYDRADIR>/<MANDANT>/custom/aip2/hytnrcfg.ini
AIP	<HYDRADIR>/<MANDANT>/custom/aip/hytnrcfg.ini
CTWIN	<HYDRADIR>/ctnet/win/ctwin/custom/hytnrcfg.ini

You need to create the INI file manually if not available.

Terminal

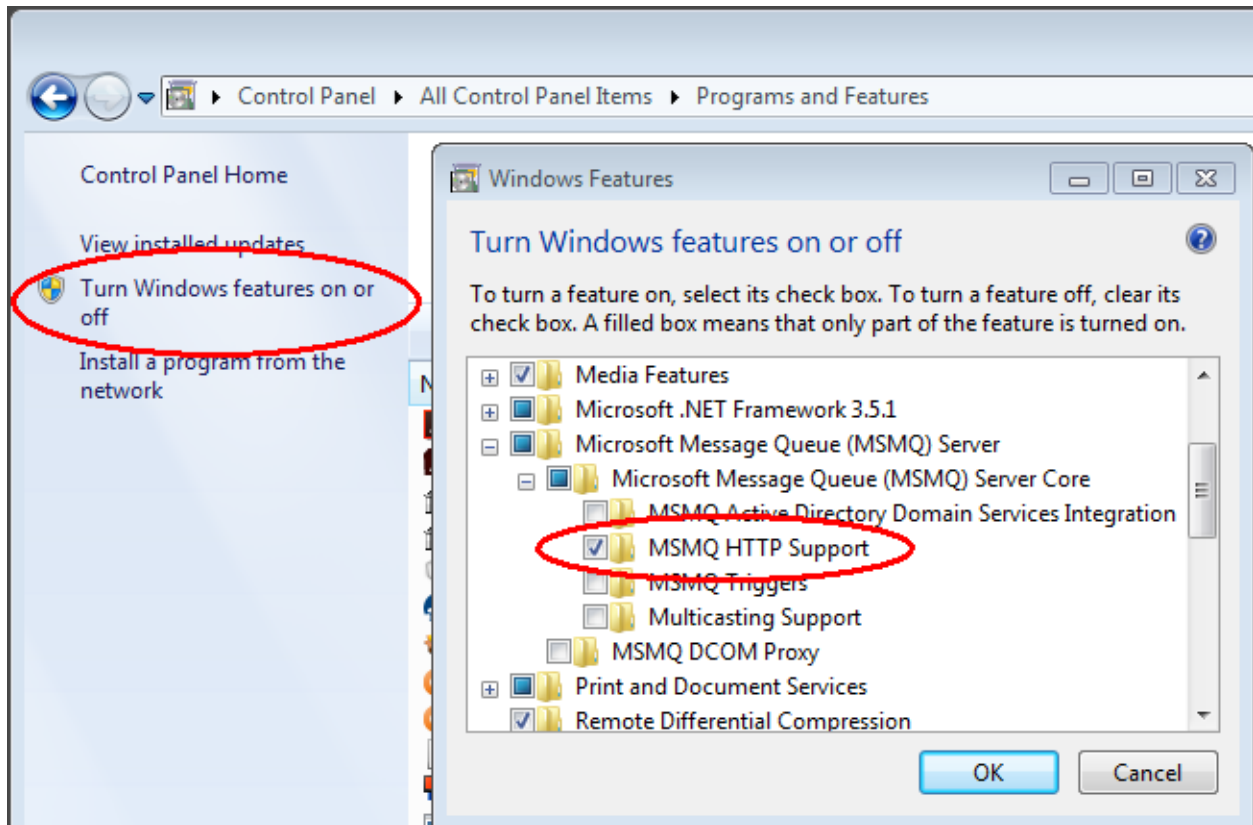
As a minimum requirement, the 2.0 NET Framework must be installed in the Windows terminal. You may only print either "new" or only "old" labels (MW2.x file ext. .qr3) at the Windows' terminal.

Mixed operation is not possible.

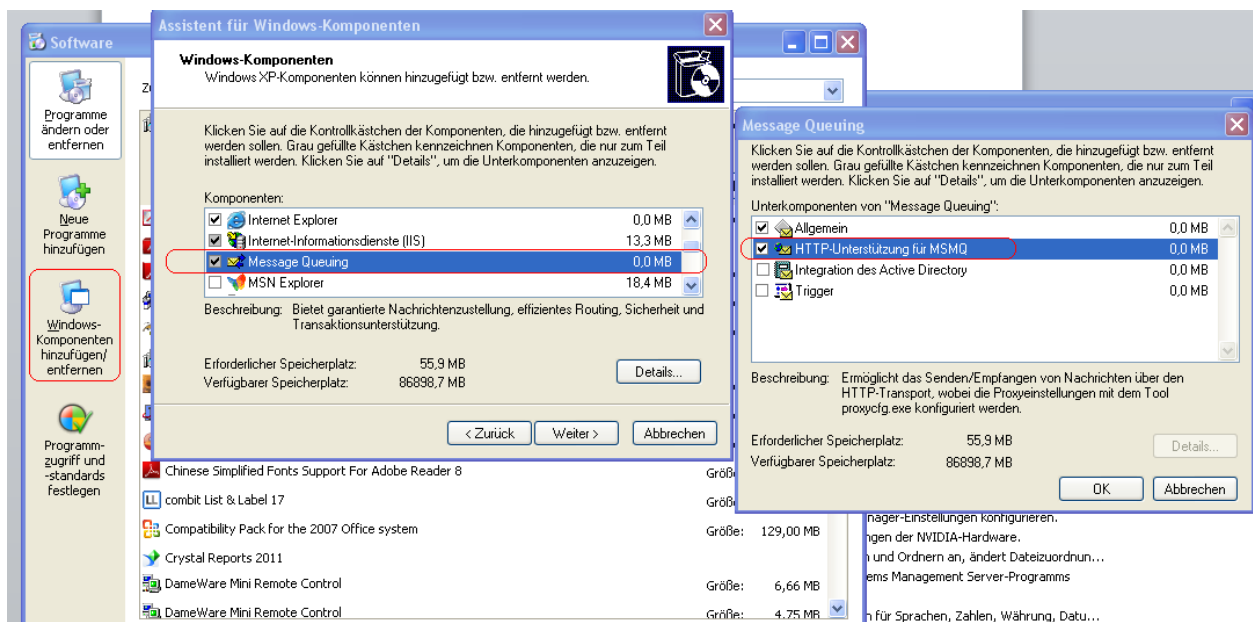
The label printer in the Windows terminal requires the following additional component:

A Windows installation medium might be required to activate the component .

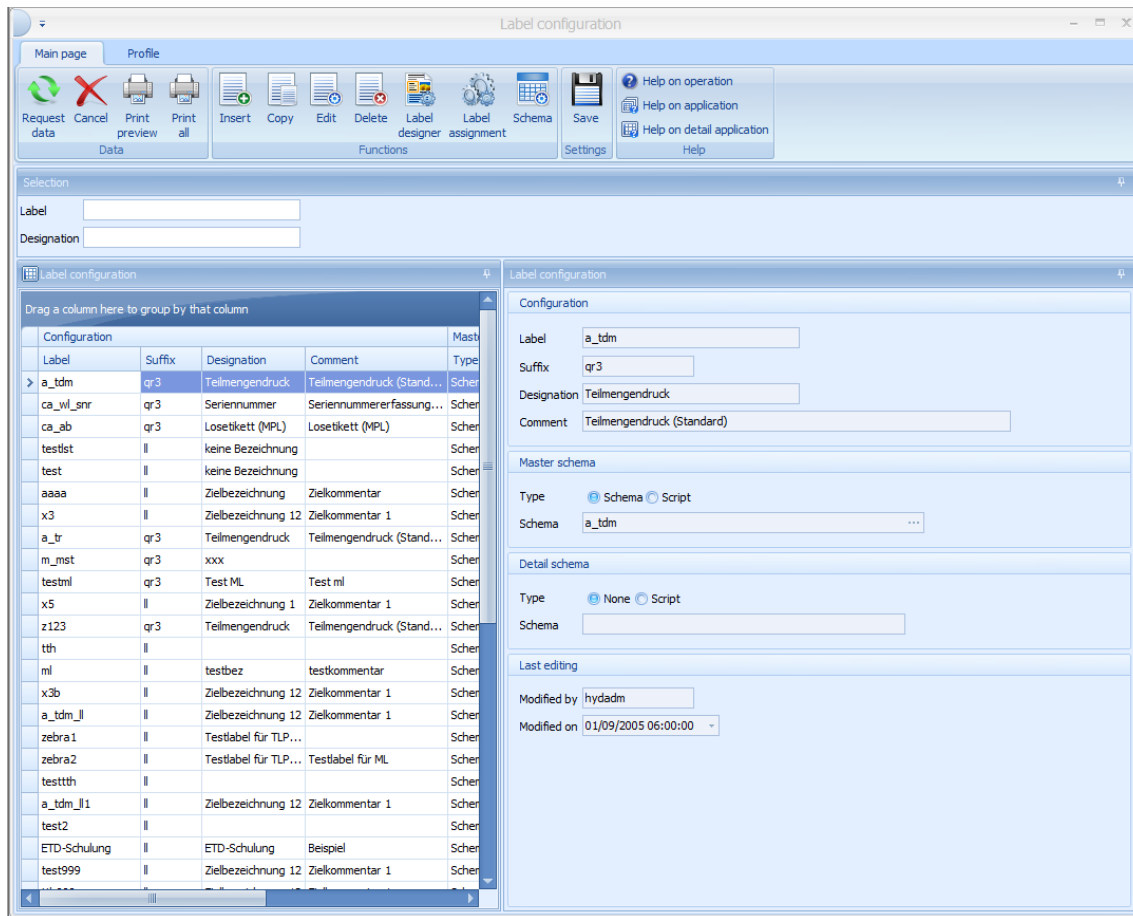
Windows 7 and new component on the window terminal



Windows XP: Component in Windows' terminal



Label configuration application



Field descriptions

Label

Label name.

Note:

The name of the label may not exceed the maximal length of 20 characters.

Suffix

File extension of label.

Please note: In MOC, only labels with the file extension ".ll" may be designed. Labels with the file extension ".qr3" cannot be designed in MOC. Such labels may only be designed in the system console. In MOC, ".qr3" labels may only be deleted.

Name

Designation for a label

Comment

Comment for a label

Master scheme**Type**

Type of stored master scheme.

Schema

Master scheme name

Detail schema**Type**

Type of stored detail scheme

Schema

Detail scheme name

Toolbar**Label designer**

Activates the integrated designer.



The MOC includes the product *List & Label* to design and execute reports and labels. The license of the integrated version can only be applied in MPDV context and not for designing and executing reports outside MPDV context.

Label Assignment

Function authorization: lblast

Schema

Schema Configuration Function authorization: lblsc

Data provision for label designer

Two data sources are available for displaying/designing the label.

→ DQMaster

→ DQDetail (optional)

Data source DQMaster

The data source DQMaster is automatically created from the available identifications in the label scheme. As per definition, the data source DQMaster consists of one data record, only. The identifications available are assigned to each master scheme in the data base. The following identifications are assigned to master scheme M_MST:

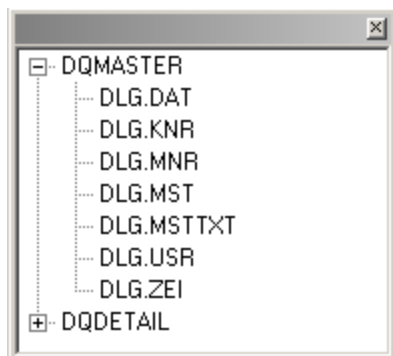
Konfiguration	
Schema	Kennung
a_smg	ANR.AGBEZ
a_smg	ANR.ANR_ANDATB
a_smg	ANR.ANR_ANZEIB
a_smg	ANR.ANR_BEARBZ
a_smg	ANR.ATK
a_smg	ANR.ATKBEZ
a_smg	ANR.AUART
a_smg	ANR.BEM1
a_smg	ANR.BEM2
a_smg	ANR.CHPFL
a_smg	ANR.DLL
a_smg	ANR.DLLKZ
a_smg	ANR.EGR:AUSB
a_smg	ANR.EGR:AUSP
a_smg	ANR.EGR:AUSS
a_smg	ANR.EGR:AUST
a_smg	ANR.EGR:DAUER
a_smg	ANR.EGR:GUTB

The identifications may be assigned to values for test purposes. This feature allows for testing labels on MOC already.

Konfiguration				
Schema	Kennung	Wert	Logging	Datentyp
m_mst	DLG.MSTTXT			
m_mst	DLG.OFF			
m_mst	DLG.USR	2100		integer
m_mst	DLG.ZEI	21600		integer
m_mst	MNR.MSDATB			datetime
m_mst	MNR.MST			integer
m_mst	MNR.MSTTXT			
m_mst	MNR.MSZEIB			integer
a_tdm	ANR.AGBEZ	Fertigung Oberteil		
a_tdm	ANR.ANR	V00000090010	ANR	
a_tdm	ANR.ANR_ANDATB			datetime
a_tdm	ANR.ANR_ANZEIB			integer
a_tdm	ANR.ANR_BEARBZ	3600		integer
a_tdm	ANR.ATK	OB-4662		
a_tdm	ANR.ATKBEZ	Oberteil OB-46		
a_tdm	ANR.AUART			
a_tdm	ANR.AUNR_ATK	Rahmen, verzinkt		

For label printing, these identifications are provided by the Windows shop floor terminal.

In the report designer, these identifications are available in the data source DQMaster.



These fields may be placed on the report in any way.



Data source DQDetail

As regards other data, the data source DQDetail may be used. This data provision is based on an additional HYDRA server script which may be executed optionally in a dialog. Each label may be assigned to a server script. The values assigned to the label as identifications are available to the script as parameters.

Important note:

The names of the Userexits must start with the prefix **u_**. The maximum permissible length of the script name comprises 20 characters.

Sample name: u_l_anr

If no Userexit is assigned to the label, the data source DQDetail is not available in the report designer.

Script - example:

In case of an order interruption, all order operations which have not yet been terminated but are already planned for a machine should be printed.

For this purpose, the following script was prepared:

```

hydra basic;

import DLG_DATA    char(30000);

long main()
{
    ret      long;
    aunr     char(40);
    aunr_len long;
    // -----
    // Tabelle(n) definieren
    // -----
    ret = Callback( "SetTables", "auftrag_status ast, auftrags_bestand ab" );

    // -----
    // Spalten definieren
    //
    //           "Spalte in Datenbank | Akronym   | Datentyp"
    // -----
    ret = Callback( "AddColumn", "ast.auftrag_nr      | ANR      | string" );
    ret = Callback( "AddColumn", "ast.prod_kenn    | PRODKENN | string" );
    ret = Callback( "AddColumn", "ab.masch_nr     | MNR      | string" );
    ret = Callback( "AddColumn", "ab.mgruppe     | MGRP     | string" );
    ret = Callback( "AddColumn", "ast.anmeldat    | DATB     | datetime" );
    ret = Callback( "AddColumn", "ast.anmeldzeit  | ZEIB     | integer" );
    ret = Callback( "AddColumn", "ast.gut_pri     | EGRGUTP  | decimal" );

    // -----
    // Clauses definieren (optional)
    // Es kann eine Where-Clause, eine optionale Group-By-Clause
    // und eine optionale Order-By-Clause definiert werden.
    // -----
    aunr      = get_bapi_val(DLG_DATA, "ANR.ANR");
    aunr_len = callback ( "HYDRA.SELECT", "LEN:AUNR" );

    ret = Callback( "SetClauses",
        "where ab.auftrag_nr = ast.auftrag_nr " ||
        "and ast.a_typ = 'AG' " ||
        "and ast.prod_kenn in ('V', 'U', 'L', 'F') " ||
        "and ast.eingeplant = 'M' " ||
        "and ab.aunr = " || BV(aunr[1,aunr_len]) ||
        "order by ast.auftrag_nr");

    // -----
    // Liste erzeugen
    // -----
    ret = Callback( "MakeList", "" );

    return ret;
}

```

This script produces the following output file:


```

ANR=string|PRODKENN=string|MNR=string|MGRP=string|DATB=datetime|ZEIB=integer|EGRGUTP=decimal|
V00000020010|E|VERSAND|Versand|04/02/2012|35307|1.000000|
V00000020020|E|VERSAND|Versand|04/02/2012|38049|1.000000|
V00000020030|E|VERSAND|Versand|04/02/2012|38486|1.000000|
V00000020040|E|VERSAND|Versand|04/04/2012|32623|1.000000|

```

Data type definition

```

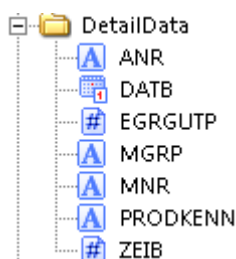
// -----
// Define columns
//
// -----
// "Column in data base |Acronym |Data type"
// -----
ret = CallBack( "AddColumn", "ast.auftrag_nr |ANR |string" );
ret = CallBack( "AddColumn", "ast.prod_kenn |PRODKENN |string" );

```

The data type definition is included in the header line for the Userexits. The following data types are supported.

Data type	Comment
string	Default - Data type. If no data type is stored, this data type is used.
integer	Numeric value without decimal places. If you want time fields to be displayed correctly, you must enter this data type for these fields.
datetime	Date values Date fields have to be generated with this data type. Correct date formatting is only possible with this data type.
decimal	Numeric value with decimal places.
void	If void is indicated, no data type is displayed. Such a field is treated as a string field.

In the label designer, this script provides the following additional data fields:



Please

note:

The field designations must not include any special characters. Permissible characters are all letters from A to Z, the underline _ and numbers 0-9.

Deployment of a label

This Section describes step by step the processes required for deploying a label to another HYDRA server. An existing label on the target server is overwritten.

It is assumed that label development is only performed on one HYDRA server (development server) within a company. On the other HYDRA servers (target servers), copies of the labels are stored.

Please note: The directory **./custom/reports** mentioned further below is located directly in the **HYDRA System Directory** on the HYDRA server.

Beispiel:

In diesem Beispiel wird ein Etikett mit dem Namen **m_mst_1** deployed.

The name of the master scheme is **m_mst**.

A script with the name **u_ett_m_mst** is assigned to this label.

The table below shows the sample data:

Description	Table	Example	Files	Optional
Label	hyd_printdesign	m_mst_1	./custom/reports/m_mst_1.ll	
Label Assignment	hyd_prndesign_cfg	m_mst_1		
Schema	hyd_prn_schema	m_mst		
Scheme parameters	hyd_prn_schema_det	m_mst		
Script	hyd_userexit	u_ett_m_mst	./custom/userexit/u_ett_m_mst*.hsc	☒

Note: However, a script only needs to be deployed if a script has been specified in the label detail scheme.

Preparations on the development server

Go to the **./custom/reports** directory on the HYDRA server.

Execute the following commands here:

```
mysql.out(exe) -r -
xunload to hyd_printdesign.lod select report, suffix, version, bezeichnung, kommentar,
schema_master, sart_master, schema_detail, sart_detail, param_01, param_02, pdouble_01,
pdouble_02, param_str01, param_str02 from hyd_printdesign where report = 'm_mst_1';

xunload to hyd_prndesign_cfg.lod select zuordnung, report, suffix, version, bezeichnung,
kommentar, etyp, protokoll, aktiviert, anzahl, key1, value1, key2, value2, key3, value3, key4,
value4, key5, value5, printmode, logdrucker, zuordnungs_art, dialog, verarbeitung, batchprint,
batchdataconv, batch, param_01, param_02, pdouble_01, pdouble_02, param_str01, param_str02 from
hyd_prndesign_cfg where report = 'm_mst_1';

xunload to hyd_prn_schema.lod select hyschema, hyschema_art, version, bezeichnung, kommentar,
param_01, param_02, pdouble_01, pdouble_02, param_str01, param_str02 from hyd_prn_schema where
hyschema = 'm_mst';

xunload to hyd_prn_schema_det.lod select hyschema, kennung, typ, wert, mdkey, protkey, param_01,
param_02, pdouble_01, pdouble_02, param_str01, param_str02 from hyd_prn_schema_det where hyschema
= 'm_mst';

xunload to hyd_userexit.lod select userexit, typ, bezeichnung, kommentar, version, aktiviert,
released, param, bearb, bearb_date, bearb_time, aktiv, aktiv_date, aktiv_time, inaktiv,
inaktiv_date, inaktiv_time from hyd_userexit where userexit = 'u_ett_m_mst';
exit
```

Copying files

Copy the following files into the relevant directory on the target server. If you load the files on the target server by using the HYDRA Install Tool, loading the file `hyd_userexit.lod` is not necessary.

Description	Example	Files
Label	m_mst_1	./custom/reports/m_mst_1.ll
LOD files	*.lod	./custom/reports/hyd_printdesign.lod ./custom/reports/hyd_prndesign_cfg.lod ./custom/reports/hyd_prn_schema.lod ./custom/reports/hyd_prn_schema_det.lod ./custom/reports/hyd_userexit.lod
Script	u_ett_m_mst	./custom/userexit/u_ett_m_mst*.hsc

Loading DB tables on the target server

Go to the **./custom/reports** directory on the target server.

```
mysql.out(exe) -r -  
load from hyd_printdesign.lod insert into hyd_printdesign;  
load from hyd_prndesign_cfg.lod insert into hyd_prndesign_cfg;  
load from hyd_prn_schema.lod insert into hyd_prn_schema;  
load from hyd_prn_schema_det.lod insert into hyd_prn_schema_det;  
load from hyd_userexit.lod insert into hyd_userexit;  
exit
```

4 Schema Configuration

Overview

Menu	System administration → Label configuration → Schema
Transaction code	Iblsch
Function authorization	Iblsc

The screenshot displays the 'Schema' configuration window. The 'Main page' tab is selected, showing a toolbar with various actions like 'Request data', 'Cancel', 'Print preview', 'Print all', 'Insert', 'Copy', 'Edit', 'Delete', 'Detailed schema', 'Save', and 'Help'. Below the toolbar, there are input fields for 'Schema' and 'Designation'. A list of schemas is shown on the left, with 'a_ab' selected. The right pane shows the configuration for 'a_ab', including Designation, Comment, Type, and Version. The 'Last editing' section shows the user 'mpdv_ml' and the date '15/05/2012 11:06:19'.

Purpose

Use the label designer to design labels via the MOC. The designer must know the available print data. Use the schema configuration for this purpose. Each label must be assigned to a schema. Detail data is assigned to the single schemas. This detail data stands for the data fields that will be used for printing. These data fields are placeholders. The Windows terminal replaces these placeholders with real data during printing. MPDV already provides over 30 pre-defined schemas. The schema name corresponds to the action to be performed via the terminal. Use the schema a_ab if you want to print a label when you log off an order. This schema includes approx. 60 data fields. The label designer can place these data fields anywhere on the label. If you need additional fields, you have to ensure that the terminal knows these fields at the time of printing. You can create additional fields as part of customizing the HYDRA system.

Integration

The label designer uses the schema configuration. Consequently, the label designer is provided with the available fields.

Requirements

Users must have the function authorization "lblsc" in order use this function. This provides the user with full access to all functions.

Function authorizations can also protect the individual editing applications. The descriptions of the corresponding editing applications specify the required function authorizations. If users are authorized for an editing application, they are automatically authorized to access the main application where they may also view data.

Field descriptions

Schema

Name of the schema. The HYDRA database already includes essential, pre-configured schemas for use with dynamic dialogs.

Designation (name)

Short description of the schema.

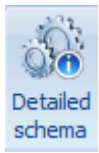
Comment

Detailed description of the schema.

Type

This identifier differentiates between master schema and detail schema. There is no technical differentiation between master and detail schema. At present, you can only configure master schemas.

Toolbar



Detail schema

Click this button to open the application "detail schema".

5 Scheme Detail Configuration

Overview

Menu	System administration → Label configuration → Detailed scheme
Transaction code	Ibldet
Function authorization	Ibldet

Detailed schema

Main page Profile

Request data Cancel data Print preview Print all Insert Copy Edit Delete Save Help on operation Help on application Help on detail application Help

Selection

Schema

Identification

Detailed schema

Drag a column here to group by that column

Configuration				Last editing		
Schema	Identification	Value	Logging	Data type	Modified by	Modified on
> a_ab	ANR.AGBEZ					
a_ab	ANR.ANR_ANDATB			datetime		
a_ab	ANR.ANR_ANZEIB			integer		
a_ab	ANR.ANR_BEARBZ	21600		integer	mpdv_ml	04/06/2012
a_ab	ANR.ATK					
a_ab	ANR.ATKBEZ					
a_ab	ANR.AUART					
a_ab	ANR.BEM1					
a_ab	ANR.BEM2					
a_ab	ANR.CHPFL					
a_ab	ANR.CNR					
a_ab	ANR.DLL					
a_ab	ANR.DLLKZ					
a_ab	ANR.EGR:AUSB			decimal		
a_ab	ANR.EGR:AUSP			decimal		
a_ab	ANR.EGR:AUSS			decimal		
a_ab	ANR.EGR:AUST			decimal		
a_ab	ANR.EGR:DAUER			decimal		
a_ab	ANR.EGR:GUTB			decimal		
a_ab	ANR.EGR:GUTP			decimal		
a_ab	ANR.EGR:GUTS			decimal		
a_ab	ANR.EGR:GUTT			decimal		
a_ab	ANR.EGR:PDauer			integer		
a_ab	ANR.EGS:GUT			decimal		
a_ab	ANR.HZBEZ					
a_ab	ANR.H7TYP					

Configuration

Schema

Identification

Value

Logging

Data type

Last editing

Modified by

Modified on

Usage

Labels are designed in MOC by using the label designer. The designer must know the available printing data. For this purpose, the scheme configuration is used. Each label must be assigned to a scheme. The individual schemes in turn are assigned to detail data. Such detail data represent the data fields for the subsequent print. They are placeholders filled with real data from the Windows terminal in the real printing process. MPDV already provides over 30 pre-defined schemes. The scheme name always corresponds to the action to be performed on the terminal. If a label is to be printed upon an order log-off, the scheme `a_ab` is to be used. The scheme comprises approx. 60 data fields which may be placed at any point on the label by the label designer. If additional fields are needed, it must be ensured that the terminal has information on these fields at the time of printing. This may, for instance, be effected through HYDRA Customizing.

Integration

The label designer uses the scheme configuration. The label designer is informed on available fields through this.

Prerequisite

In order to be able to use this function, the data base patch (`db_sql\dbp_labeldesign.hsc`) is required.

In order that the user may apply this function, he/she must have the function authorization "Ibldet". This provides the user with full access to all functions.

Individual maintenance applications may also be protected by function authorizations. These are identified in detail in the description of the relevant maintenance application. If the user is authorized to perform a maintenance application, he/she is automatically authorized to access the main application where he/she may also view data.

Field descriptions

Scheme

Scheme name

ID

HYDRA ID

Value

Exemplary values may be stored here. Such values are only adopted for the label as examples in the label preview on MOC.

Logging

The key stored here is available as a filter parameter in the reprint screen. The terminal number, date and time are identified implicitly.

The following values are stored:

Recording	Comment
MNR	Workplace
ANR	Complete order number
CNR	Batch no.

Data type

This is where the data type of the field may be stored. If no data type is stored, the field is automatically processed as a "string" field.

The following data types are supported:

Data type	Comment
string	Default - Data type. If no data type is stored, this data type is implicitly used.
integer	Numeric value without decimal places. In order that time fields may be displayed correctly, this data type is to be used for these fields.
datetime	Date values. Date fields have to be generated with this data type. Correct date formatting is only possible with this data type.
decimal	Numeric value with decimal places.

Available report fields from collection

This Section describes the IDs available in the relevant dialogs at the Window shop floor terminal. These IDs are already assigned to the delivered master data schemes. The relevant master data schemes are designated according to the dynamic dialogs.

Example Machine status change:

Dynamic dialogs	Master scheme
M_MST	m_mst

Prefix DLG

All input and/or status fields of the dynamic dialog are configured and/or defined with the prefix <DLG>.

ID	Comment
DLG.OFF	Offline - ID {Y/N} Y = the triggering message was stored in the queue. This is the case if no network connection to the HYDRA server exists or if a record is already in the queue.
DLG.DLGCFG	Dynamic dialog ID for label print
DLG.DLG	Dynamic dialog sending ID
DLG.DLGDAT	Start date of dynamic dialog
DLG.DLGZEI	Start time of dynamic dialog
DLG.DAT	Posting date of event
DLG.ZEI	Posting time of event
DLG.MNR	Machine
DLG.MST	Machine status
DLG.MSTTXT	Machine status text
DLG.ANR	Order
DLG.ATK	Article
DLG.ATKBEZ	Article designation
DLG.AGBEZ	Operation designation
DLG.AART	Order type (ANR.AUART)
DLG.KNR	Card number of reporting person
DLG.PNR	Personnel number of reporting person
DLG.BPOS	Operator position
DLG.CNR	Batch no. (new = for starting message/label)
DLG.CNR:AUS	Batch no. (old = for terminating message/label) Please note: (only available if actually existing in dynamic dialog, otherwise please refer to <ANR.CNR>)
DLG.NUMPRN	The dynamic dialog field <NUMPRN> is used to overdrive the number of labels to be printed as configured in the label assignment. Possible values: - < 0 Printing job is not executed - 0 Printing job is executed with the configured number - > 0 Printing job is executed with the entered number. The requested functionality on the terminal is achieved by targeted configuration of the input field <NUMPRN> of the relevant dialog in the dynamic dialog configuration.
...	For additional information, please refer to documentation "SCS-PDM". <u>Please note:</u> The IDs used there are extended by the prefix <DLG> for identification.

Prefix MNR

If <MNR = DLG.MNR> is defined in the reported dialog/event, the relevant machine line is read from the local list MNR.LST and transferred to the printing program with prefix <MNR.>. The data correspond to the current values excluding the changes triggered by the event.

Please note:

Such extended machine information may possibly not be established locally on an ADE shop floor terminal without any fixed Machine - Terminal - Assignment. This, for instance, applies for a label print in the event "Log on order" (A_AN*,A_P_AN*), if no order log-on has taken place yet at this terminal without any fixed/configured Machine - Terminal - Assignment. As a standard, the information to be printed may be loaded and printed as detail data via "HYDRASCRIP".

ID	Comment
MNR.MNR	Machine
MNR.MGRP	Machine group
MNR.MBEZK	Designation
MNR.MBEZL	Explicit machine designation
MNR.KST	Cost center
MNR.MST	Status
MNR.MSTTXT	Status text
MNR.PKENN	Production identifier
MNR.MSDATB	Start date for status
MNR.MSZEIB	Start time for status
MNR.SKNR	Shift number
MNR.SKDATB	Shift start date
MNR.SKZEIB	Shift start time
MNR.SKDATE	Shift end date
MNR.SKZEIE	Shift end time
MNR.AGR:GUT	Shift-related machine yield quantity
MNR.AGR:AUS	Shift-related machine scrap quantity
MNR.AGR:HUB	Shift-related strokes
MNR.TLG	Current machine partitioning
MNR.SZY	Current target cycle number
...	For additional information, please refer to documentation "BDE-PDM", Section "<i>Machine Information</i>" <u>Please note:</u> The IDs used there are extended by the prefix <MNR> for identification.

Prefix ANR

If <ANR = DLG.ANR> is defined in the reported dialog/event, the relevant order line is read from the local list ANR.LST and transferred to the printing program with prefix <ANR.>. The data correspond to the current values excluding the changes triggered by the event.

Please note:

In case of a label print for the event "Order log on" (A_AN*,A_P_AN*), the extended order information is usually not available locally. As a standard, the information to be printed may be loaded and printed as detail data via "HYDRASCRIP".

ID	Comment
ANR.ANR	Order number total
ANR.AUNR	Order number
ANR.AGNR	OP number
ANR.AFOLG	Sequence
ANR.UAGNR	Sub-OP number
ANR.SPLNR	Split number
ANR.AUART	Order type
ANR.MNR	Planned individual machine
ANR.MGRP	Machine group
ANR.ATK	Article
ANR.ATKBEZ	Article designation
ANR.AST	Order status
ANR.ASTTXT	Status text
ANR.CNR	Batch no.(old = terminating posting)
ANR.DLL	Run-through batch number (if <> with ANR.CNR) ➔ otherwise batch number (old = terminating posting)
ANR.TLG	Partitioning
ANR.SZY	Target cycle
ANR.BEM1	Note1
ANR.BEM2	Note2
ANR.HZTYP	Semi-finished article type
ANR.HZBEZ	Semi-finished article design.

ID	Comment
....	For additional information, please refer to documentation "BDE-PDM", Section "Order list". Please note: The IDs used there are extended by the prefix <ANR> for identification.

Prefix PRN

ID	Comment
PRN.REPRNCNT	Reprint counter. This field is used to identify the reprint. For each reprint, this counter is incremented by value 1. In the label, the condition <code>STRTONUM(DQMaster.PRN.REPRNCNT) > 0</code> may be used to specifically identify a reprint. In HYDRA scripts, the following condition may be used for inquiries: <pre>if (get_bapi_val(DLG_DATA, "PRN.REPRNCNT") > 0) { }</pre>

Prefix RET

ID	Comment
RET.RET	Return code of command
RET.KT	Command short text
RET.LT	Command long text
RET.*	The ID RET may be used to address all values returned by the HYDRA server during command posting. The return values must be extended by the prefix <RET>. Please note: These values are not available if the system is offline.

Machine-related dialogs (STD)

Dialog M_MST

Description: Machine status change (default)

ID	Comment
DLG.*	Cf. prefix DLG
MNR.MST	Machine target status for terminating posting

ID	Comment
MNR.MSDATB	Machine status start date
MNR.MSZEIB	Machine status start time
MNR.MSDAUER	<i>Non-current period since last machine status change (is not updated locally, usually shows period from start time to latest list update)</i>

Dialog M_SZY

Description: Change machine target cycle (default)

ID	Comment
DLG.*	Cf. prefix DLG
MNR.SZY	Machine target status for terminating posting

Dialog M_TLG

Description: Change machine partitioning (default)

ID	Comment
DLG.*	Cf. prefix DLG
MNR.TLG	Machine partitioning for terminating posting

Personal postings (STD)

Dialog P_AN

Description: Log person on (default)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog P_AB

Description: Log person off (default)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog P_AAB

Description: Log all persons off (default)

ID	Comment
DLG.*	Cf. prefix DLG

Order-related postings (STD)

Dialog A_AN

Description: Log order on (default)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog A_P_AN

Description: Log order and person on (default)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog A_TR

Description: Partial confirmation/upload (default)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog A_UN + also for (CHV)

Description: Interrupt order (default)

ID	Comment
DLG.*	Cf. prefix DLG
ANR.CNR	In module (CHV) Batch number (old = terminating posting)

Dialog A_AB + also for (CHV)

Description: Log order off (default)

ID	Comment
DLG.*	Cf. prefix DLG

ID	Comment
ANR.CNR	In module (CHV) Batch number (old = terminating posting)

Dialog A_SMG

Description: Change order target quantity (default)

ID	Comment
DLG.*	Cf. prefix DLG

Order-related postings (CHV)

Dialog A_AN_CHV

Description: Log batch order on (CHV)

ID	Comment
DLG.*	Cf. prefix DLG
DLG.CNR	Batch number (new = starting posting)

Dialog A_P_AN_CHV

Description: Log batch order and person on (CHV)

ID	Comment
DLG.*	Cf. prefix DLG
DLG.CNR	Batch number (new = starting posting)

Dialog CA_WL

Description: Batch change (CHV)

ID	Comment
DLG.*	Cf. prefix DLG
DLG.CNR	Batch number (new = starting posting)
ANR.CNR	Batch no.(old = terminating posting)

Order-related postings (MPL)

Machines with batch management

→ Batch tracing (input/output batches)

→ Run-through batch processing

Dialog A_AN_MPL

Description: Log order on (MPL)

ID	Comment
DLG.*	Cf. prefix DLG
DLG.CNR	Batch number (new = starting posting)

Dialog A_P_AN_MPL

Description: Log order and person on (MPL)

ID	Comment
DLG.*	Cf. prefix DLG
DLG.CNR	Batch number (new = starting posting)

Dialog CA_WL_MPL

Description: Output batch change in module MPL

ID	Comment
DLG.*	Cf. prefix DLG
DLG.CNR	Batch number (new = starting posting)
ANR.CNR	Batch no.(old = terminating posting)

Dialog A_UN_MPL

Description: Interrupt order (MPL)

ID	Comment
DLG.*	Cf. prefix DLG
ANR.CNR	Batch no.(old = terminating posting)

Dialog A_AB_MPL

Description: Log order off (MPL)

ID	Comment
DLG.*	Cf. prefix DLG
ANR.CNR	Batch no.(old = terminating posting)

Batch-related postings (MPL)

Dialog C_GEN

Description: Collect goods receipt batch (MPL)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog C_MR

Description: Weigh batch (MPL)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog C_UMB

Description: Repost batch (MPL)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog CE_AN

Description: Log input batch on (MPL)

ID	Comment
DLG.*	Cf. prefix DLG

Dialog CE_AB

Description: Log input batch off (MPL)

ID	Comment
DLG.*	Cf. prefix DLG

Special dialogs

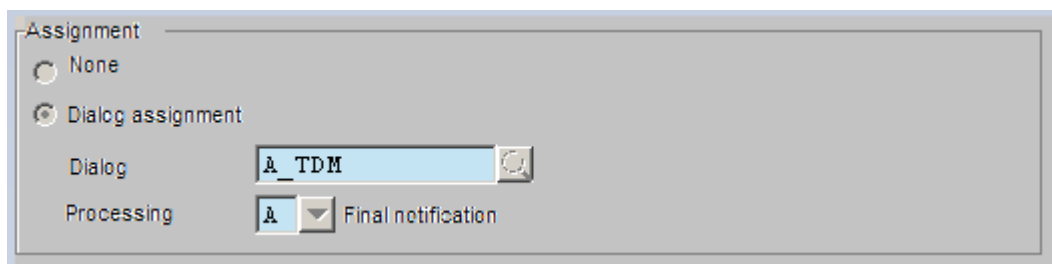
Order-related postings

Partial quantity documentation A_TDM

Description: Additional function Partial quantity documentation

The label *a_tdm* is an integral part of the delivery data of a HYDRA system. This label is used for printing partial quantities on the HYDRA Windows terminal. Label assignment is not available for this label. If the terminal is configured in accordance with documentation [AIP-TDM](#), an automatic label assignment is generated for the relevant terminal. Manual label assignment will overdrive automatic assignment. Consequently, it is also possible to configure a label reprint for this posting. The standard label *a_tdm* should not be deleted.

As regards manual label assignment, care should be taken to store the value A_TDM in dialog assignment.



ID	Comment
ANR.*	Cf. prefix ANR

6 Label reprint

Overview

Menu	System administration → Label configuration → Label configuration
Transaction code	lblpr
Function authorization	lblpr

Label reprint

Main page Profile

Request data Cancel Print preview Print all Label reprint Save Help on operation Help on application Help on detail application

Selection

Terminal From 10/05/2012 00:00:00 To 21/06/2012 23:59:59

Workplace

Operation

Batch no.

Label reprint

Drag a column here to group by that column

Label assignment	Label	Assignment	Id	Terminal	Workplace	Operation	Batch no.	Date	Time
A_TR	zebra2.ii	a_tdm_II1	100	30	VERSAND	V00000020050		29/05/201...	15:23:23
A_TR	zebra2.ii	a_tdm_II1	101	30	VERSAND	V00000020050		29/05/201...	15:49:19
A_TR	zebra2.ii	a_tdm_II1	102	30	VERSAND	V00000020050		29/05/201...	15:52:35
A_TR	zebra2.ii	a_tdm_II1	103	30	VERSAND	V00000020050		29/05/201...	15:54:34
A_TR	zebra2.ii	a_tdm_II1	104	30	VERSAND	V00000020050		29/05/201...	15:58:21
A_TR	zebra2.ii	a_tdm_II1	105	30	VERSAND	V00000020050		29/05/201...	15:58:49
A_TR	zebra2.ii	a_tdm_II1	106	30	VERSAND	V00000020050		30/05/201...	10:59:39
A_TR	zebra2.ii	a_tdm_II1	107	30	VERSAND	V00000020050		30/05/201...	16:51:48
A_TR	zebra2.ii	a_tdm_II1	108	30	VERSAND	V00000020050		30/05/201...	16:52:44
A_TR	zebra2.ii	a_tdm_II1	109	30	VERSAND	V00000020050		30/05/201...	16:53:17
A_TR	zebra2.ii	a_tdm_II1	110	30	VERSAND	V00000020050		30/05/201...	16:53:36
A_TR	zebra2.ii	a_tdm_II1	111	30	VERSAND	V00000020050		31/05/201...	7:24:37
A_TR	zebra2.ii	a_tdm_II1	112	30	VERSAND	V00000020050		31/05/201...	9:42:52
A_TR	zebra2.ii	a_tdm_II1	113	30	VERSAND	V00000020050		31/05/201...	12:07:43
A_TR	zebra2.ii	a_tdm_II1	114	30	VERSAND	V00000020050		31/05/201...	12:20:11
A_TR	zebra2.ii	a_tdm_II2	115	30	VERSAND	V00000020050		31/05/201...	12:20:11
A_TR	zebra2.ii	a_tdm_II1	116	30	VERSAND	V00000020050		31/05/201...	12:21:44
A_TR	zebra2.ii	a_tdm_II2	117	30	VERSAND	V00000020050		31/05/201...	12:21:44
A_TR	tthmtest.ii	tthtest2	118	30	VERSAND	V00000020050		31/05/201...	14:10:45
A_TR	tthmtest.ii	tthtest2	119	30	VERSAND	V00000020050		01/06/201...	7:23:42
A_TR	tthmtest.ii	tthtest2	120	31	TEST_M1	V00000050010		01/06/201...	10:59:52
A_TR	tthmtest.ii	tthtest2	122	31	TEST_M1	V00000050010		01/06/201...	11:01:57
A_TR	tthmtest.ii	tthtest2	124	31	TEST_M1	V00000050010		01/06/201...	11:08:38

Purpose

Select the option "Log" (protocol) in the *Label assignment* application to log the printout in the HYDRA database. This logged data is available for reprinting the label. Only the data of the master schema is logged. The system uses this data to complete the master schema for the reprint. If a script is assigned to the printed label, the script is also executed during reprinting. The script is parametrized with the logged data and executed on the HYDRA server. The detail data is not logged.

The keys defined in the *Logging* column of the master schema are available as filter parameters in the *Label reprint* application. The system implicitly determines the terminal number, date and time.

Integration

The MOC only reprints labels with the file extension ".ll". The MOC does not reprint labels with the file extension ".qr3".

Requirements

You need the function authorization "lblr". The function authorization "lblr" grants full access to all functions.

Function authorizations can also protect the individual editing applications. The descriptions of the corresponding editing applications specify the required function authorizations. If you are authorized for an editing application, you are automatically authorized to access the main application where you may also view the data.

Field descriptions

Dialog

Action that triggered label printing.

Label

Label name.

Assignment

Name of the label assignment.

ID

Unique ID of the print job.

Terminal

Number of the terminal where the label was printed.

Workplace

Workplace number where the label was printed.

Please note: The field is only completed if the defined label schema includes a data record with the entry "MNR" in the *Logging* field of the details.

Operation

Complete order number

Please note: The field is only completed if the defined label schema includes a data record with the entry "ANR" in the *Logging* field of the details.

Batch (lot)

Batch number

Please note: The field is only completed if the defined label schema includes a data record with the entry "CNR" in the *Logging* field of the details.

Toolbar

Label reprint

Opens the preview with the reprint data.

Reprinting via the terminal

You can only use the reprint function if the Windows shop floor terminal is online.

Use the MES Development Suite function *MDS-ETD* to configure the terminal reprint function as a customization. The document MDS-ETD outlines this customization in a separate section.

Data retention on the HYDRA server

The data for the label reprint is available on the HYDRA host computer for 15 days. If required, you can change this period in the data management (product ETD, object PRN_LOG).

7 Label reprint via the terminal: configuration

Overview

Usually, you reprint labels via the MOC. If you want to reprint labels at the shop floor terminal, you have to configure this option as part of system customization.

Requirements

- Enable the dynamic dialog <EV_NDRUCK> for the system and the terminal.
- Save printed labels in the server in order to be able to reprint these labels. To do so, enable the option "log" for the relevant label in the label assignment.
- Configure the dialog to reprint labels in the terminal. See the description below.

AIP2 configuration

Button starting reprinting

In the AIP2 tile view, use the dialog EV_NDRUCK with the <OnClick> property to configure a button or tile starting the reprint process.

- Example for a button in the main view:


```
<!--Re-print-->
<element class="TGUIButton">
  <Align>alTop</Align>
  <AlignWithMargins>true</AlignWithMargins>
  <BorderWidth>5</BorderWidth>
  <Height>50</Height>
  <Margins>
    <Top>0</Top>
    <Left>0</Left>
    <Right>10</Right>
    <Bottom>7</Bottom>
  </Margins>
  <Alignment>taLeftJustify</Alignment>
  <Color Define="COLOR_MENU">$E0E0E0</Color>
  <Caption Function="Translate" LanguageKey="lkRePrint">Nachdruck</Caption>
  <OnClick Identifier="EV_NDRUCK" Parameterprocessor="TFocusedDataRows">Notify</OnClick>
</element>
```

Dialog with list

- The section "Layout->reprint" of the file *ctaipay.ini* configures the layout of the list.
- Define and enable the dynamic dialog <EV_NDRUCK> for the terminals where you want to use this function.
- Add the field "number of copies" with the ID <NUMPRN> to the dialog <EV_NDRUCK>. Use this field to overwrite the number of copies that is configured by default.

Enter 0 to print the configured number of labels.

Enter a value > 0 to ignore the configured number and to print the entered number of labels.

Event-Nachdruck <EV_NDRUCK>

Arbeitsplatz: R000

Arbeitsgang: M-RF00010010

Zeitpunkt Beginn: 11.10.2017 08:05:06

Zeitpunkt Ende: 12.10.2017 08:05:06

Datum	Zeit	Dialog	Maschine	Auftrag	Verweis
02.06.06	09:08:38	CA_WL_MPL	R000	M-RF00010010	310824
02.06.06	10:07:57	CA_WL_MPL	R000	M-RF00010010	311142
02.06.06	12:13:13	CA_WL_MPL	R000	M-RF00010010	311560

Abbrechen Liste Nachdruck

12.10.17 08:05:16

Illustration: Selection list for reprinting

Initialization and processing of the dialog:

- By default, the field "workplace" includes the machine you selected in the main view. Use the selection list to choose another machine of the terminal.
- By default, the field "operation" includes the operation you selected in the main view. Use the selection list to choose another operation of the terminal. In this case, the machine pertaining to this new operation is automatically entered in the "workplace" field.
- The field "start time" is assigned to the current point in time minus 24 hours and the field "end time" is assigned to the current point in time. Add the following entry to the customer-specific configuration file <hytnrcfg.ini> to change this period:

```
[Dialog->Initialization 0/2xxx]
EV_NDRUCK-INITIALIZE-TIMESTEP=86400
```

- Click the "list" button to update the list with the entered selection criteria.

Possible selection criteria:

MNR	- Workplace/machine
ANR	- Operation
CNR	- Batch number
TNR	- Terminal number
DATB / ZEIB	- Start time
DATE / ZEIE	- End time

- Click the "reprint" button to reprint labels for the entry selected in the table.
- Click "cancel" to cancel the process.

CTWIN and AIP configuration

Button starting reprinting

Assign the entry "EV_NDRUCK,Nachdruck" to a button in the file *ctwinbut.ini/ctaipbut.ini* to enable the function for the HYDRA terminal.

Dialog with list

- The section "Layout->reprint" of the file *ctwinlay.ini/ctaipplay.ini* configures the layout of the list.
- The configuration of the dialog corresponds to the configuration in AIP2.

8 List + Labels configurations

Purpose

You can use *List & Label* to configure and/or define reports and labels. You can also use formulas in *List & Label* to specify how data is displayed and to calculate values.



The manual describing the integrated *Designer* is included in the training documents.



The MOC includes the product *List & Label* to design and execute reports. You can only use the license of the integrated version with the MOC. You cannot use this version to design and execute reports without the MOC.

Formula to display the print date in the 12-hour representation (ETD)

Example:

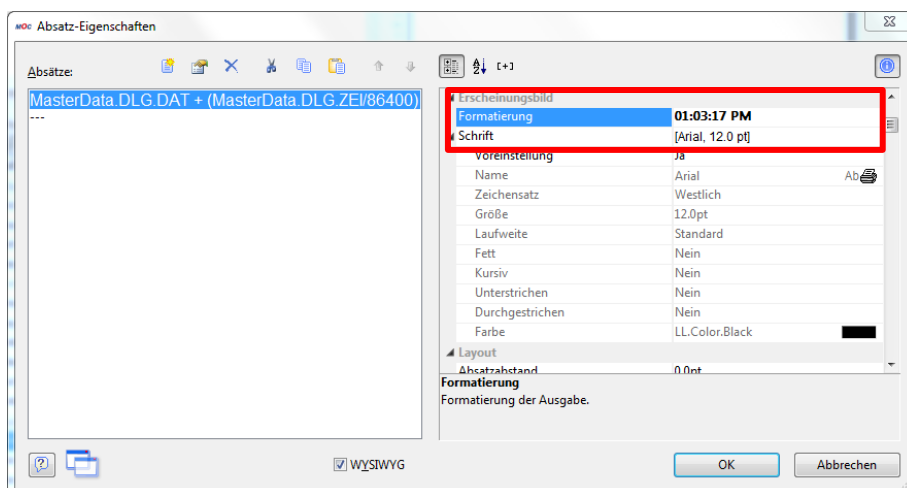
Display the print date in the 12-hour representation on the label. Since it is not sufficient to just define the formatting, you can use the following formula as an example. (Only relevant for label printing).

Solution:

Use the following formula to define the print date:

MasterData.DLG.DAT + (MasterData.DLG.ZEI/86400)

Then select the date format:



9 Central Configuration File hytnrcfg.ini

This file includes different configurations for all or single terminals at a central place.

Each section is available in a generally accepted version

[section 0].

However, entries included in this section can be overwritten by entries in a terminal-specific section [section <TNR-USER>]

<TNR-USER> = HydraUser = Terminal number + 2000 e.g. 2010,2101,..) for exactly one terminal/HYDRA User



The hytnrcfg.ini file is loaded from the server every time the terminal is started.

Section / Entry	Comment
	→→→
[Tnr configuration 0]	
FollowExternStatus=on	Transfer of machine statuses when reloading machine list Useful if status change is set by PDM or another terminal
[Terminal->Installation 0]	
InstallFonts=on	If this is set to "off" fonts will not be installed during the restart. ON=DEFAULT
OnlyInstallFontsAfterDownload=false	"InstallFonts=on": If true then fonts will only be installed directly after a download. If false then fonts will be installed every time the terminal is restarted. (false = DEFAULT)
InstallTvicport=on	If "off" the LPT driver "tvicport.sys" will not be installed. It is required for HYDRA-ZKS. ON = DEFAULT
[Terminal->USR 0]	

AttachedApplication=First	This configuration checks whether or not an application is connected in Windows that matches the file extension of the document to be displayed from the OP info dialog. If there is such an application, it will be used for displaying the document. If there is no connection, viewers configured in ctaip.ini (→ [ext. software]) and internal viewers will be used. In case, an extension is completely unknown it is attempted to display it as text Different settings may be configured: First → search for connected application first AfterUserViewer → If a UserViewer is configured this one overrides the connected application (also applies for ExcelViewer, WordViewer and PowerpointViewer) Last → Only if no ctaip.ini assignment is found for the file extension, then the connected assignment will be searched for (default). Off → Connected application is never searched.
HTTPBrowser=standard	Viewing of documents (via OP info): If documents are configured with a path of the <u>type</u> "http", the file will not be downloaded to the terminal, but the link will only be transferred to a browser. The default browser for the terminal is htmview3.exe, as this one can be operated by touchscreen. If this entry is set, the default browser configured in Windows will be used.
SupressErrorMessage=70012	Suppress message "material is not planned"
[SignatureRecording->User 0]	
ManualBadgeInput=true	This configuration specifies whether or not the field "user" can be edited in the terminal (by default: no editing) true → activates keyboard input for the "user" field in the terminal

Transparency=255	The signature dialog can also be transparent. 255 → Signature dialog is 0 % transparent (not transparent) 1 → Signature dialog is 99% transparent (maximum transparency) (Default = 155) Available as of CTAIP (V# 2.0.2.25) / CTWIN (V# 7.2.5.99)
ShowPosition=TR	The position of the signature dialog can be adjusted as follows: TL Top – Left TM Top – Middle TR Top – Right ML Middle – Left MM Middle – Middle (Default) MR Middle – Right BL Bottom – Left BM Bottom – Middle BR Bottom – Right Available as of CTAIP (V# 2.0.2.25)
USE_SERVICE_ACCOUNT=1	0 (default) SSO: do not use ServiceAccount (requires the terminal to be started with the "user" domain (SSO)). Please note: ServiceAccount=1 can only be used if all users are in the "root" domain. SubDomain users are not supported.
SIGNATURE_1_USER_TYPE=REPORTING_USER_READONLY	REPORTING_USER_READONLY The tab identifying users via the Windows user is activated and assigned to "user" by default. The "user" field is read-only. This requires, however, that in the HYDRA HR master the "SSO" option is set for all users logging in. Otherwise, successful authentication is impossible. REPORTING_USER_CHANGEABLE The tab identifying users via the Windows user is activated and assigned to "user" by default. The "user" field can be modified. This requires, however, that in the HYDRA HR master the "SSO" option is set for all users logging in. Otherwise, successful authentication is impossible.

SIGNATURE_1_LOGON_TYPE=HYDRA	"" / Not set / "EMPTY" There is also an alternative login procedure. HYDRA The tab identifying users via the Windows user is blocked. The HYDRA user must be used for identification purposes. This requires, however, that in the HYDRA HR master all users logging in are created and that the "SSO" option is not set. Otherwise, successful authentication is impossible. ACTIVEDIRECTORY The tab identifying users via the HYDRA user is blocked. The Windows user must be used for identification purposes. This requires, however, that in the HYDRA HR master the "SSO" option is set for all users logging in. Otherwise, successful authentication is impossible. MIXED_BUT_UNIQUE Either the HYDRA or Windows login procedure is available, subject to whether or not the "SSO" option is set for the registered user in the HYDRA HR master. "SSO" enabled → Windows only "SSO" disabled → HYDRA only
SIGNATURE_2_LOGON_TYPE=HYDRA	Identical to SIGNATURE_1_LOGON_TYPE (see above)
ExtendedSignatureRecording=true	Used for signatures with the terminal in the area of quality data collection.

9.1 Layout configuration

Entry		Comment
Section and/or	[terminal configuration 0] [terminal configuration 2XXX];	(general configuration) (2XXX terminal-specific configuration)
AUTO-CONFIRM-UHR-ERROR-MESSAGE=TRUE		In case of an error in reading the clock (e.g. after coming out of standby mode), this configuration makes sure that the time is accepted without having to confirm a dialog. Afterwards the terminal time will be synchronized with the server time using a PDM command.
SUPPRESS-MAXIMUM-NUMBER-OF-MACHINES-WARNING=ON		As of ctaip V# 2.0.2.23 Prevents the warning after restarting the terminal if more than 32 machines are assigned to the terminal (static/dynamic). (Default = OFF)

Entry	Comment
NetRuntimeMode=2	As of ctaip V# 2.0.2.50: Alternative calculation of the target quantity since logon: The net run time is not calculated from the times when the production lock is enabled (PSperre=green) but only from the shift times less the shift breaks. Consequently, it can also be displayed, even if the terminal program has been restarted.
Section [QRD-PRINTER->TICKET 0] ;(general configuration) [QRD-PRINTER->TICKET 2xxx] ;(2XXX configuration for a specific terminal)	
COMPLETE-ABSENCE-OF-LOCAL-MNR- DATA-FOR-EVENT=< Events >	Reloads the machine row for the configured <Events>, if it is not available locally => This configuration might be required/necessary for a group workplace without machine assignment.
COMPLETE-ABSENCE-OF-LOCAL-ANR- DATA-FOR-EVENT=< Events >	Reloads the order row for the configured <Events>, if it is not available locally ➔ This option has been implemented to access order data within the master data, e.g. when logging an order on.
COMPLETE-...-EVENT=< Events > COMPLETE-...-EVENT=#ALL# COMPLETE-...-EVENT=A_AN A_P_AN	Explanation on the configuration of <Events> ➔ Using <#ALL#> the row (ANR/MNR) that is not available is reloaded for any event. ➔ <A_AN A_P_AN> restricts reloading of information to the specified events. The ID <DLGFAM> is preferred to the ID <DLG> in order to identify the <Event>.

10 Creation of data source DQDetail with HYDRA script

HYDRA script is a tool for creating program components which do not necessarily have to be processed by the MPDV software development, but can also be created and/or adopted by trained MPDV consultants and by customers themselves.

These program components are also called "userexits".

These program components are integrated at an appropriate location in the MPDV software and hence allow for changing, overwriting or otherwise influencing customer-specific calculations and processing of the HYDRA standard.

For this purpose, values defined by the activating MPDV software are transferred to the script, and after execution of the script, defined values are retrieved again.

The script language has a syntax which is largely similar to the programming language C. Some exceptions are due to the close interaction with the databases used for HYDRA and the purpose required.

It is possible to define lists via HYDRA script. In this regard, the complete definition is made in HYDRA script. These lists are used in connection with the HYDRA production data manager HYD-PDM and HYDRA label printing.

The definition of such lists via HYDRA script has two implementation levels:

1. Creation of a basic frame

By indicating the required table(s) and columns, this basic frame provides the convenient option of exporting the contents of HYDRA tables into a PDM list very easily.

2. Extended options

By using SQL statements and the complete control functions in HYDRA script, any type of complex calculations and intermediate processing stages are possible as regards list creation.

The options are explained in the sections below by way of an example. In this example, the persons in the HYDRA HR master data are to be read as a list.

Please observe the definitions for name spaces and the other requirements described previously!

10.1 Creation of the HYDRA script

For the purpose of defining a BAPI, a script is created by means of a text editor.

- This script has the name of the list, e.g. "u_l_persons1"
- and is saved in \hydra\<system>\custom\userexits.

General details of the HYDRA script language are described in a separate document.

Import variables

Import variables

Parameter	Type	Contents
DLG_DATA	C30000 (max.)	This variable includes the dialog string. It is possible to read individual fields from this dialog string using the function get_Bapi_Val(DLG_DATA, "<Acronym>"); . These fields can then be used for specifying a Where clause for the selection of the data to be displayed in a list.

Please note:

In label printing, the data transferred in DLG_DATA matches all fields configured in the executing dialog.

Script function long main()

Parameters

none

Return value

The return value of the function main() is returned as the result of the dialog for list creation. It must be a long value:

0 : everything OK

otherwise: error code, e.g. SQL error code. For more information, please refer to the section on troubleshooting.

Explanations

The main() function controls the entire processing of list creation. Other functions are not required in the script. (However, other functions may be defined for use within the script, if required.)

Callback function long "SetTables"

Parameters

SQL fragment with table name(s). If several tables are indicated, these should be given an alias.

Return value

The return value is irrelevant, the function will always return 0.

Explanations

This function is used for the variant with the basic frame. This function is used to indicate the name of the table(s) to be listed, and to set an internal buffer with the table name(s). This/these table name(s) is/are subsequently inserted in an SQL statement.

Example

```
ret = CallBack( "SetTables", "personalstamm p, outer kostenstellen kst" );
```

Callback function long "AddColumn"

Parameters

The parameters included in this function are three values separated by "|" (Pipe).

- 1) SQL fragment including the column to be selected from the database. This is where table aliases and column aliases may be defined.
- 2) Acronym for the list header. The acronym can be left empty here; the column name and/or the column alias indicated in 1) is then used as the acronym.
- 3) Header designation. The requirement of a designation depends on you external PDM application. **PLEASE NOTE:** Specifying the identifier "DCD=N|" will have the effect that only the acronyms and no designations are output in the header when the list is created. (available as from hymw 7.2.1.75 June 2005)

Return value

The return value is irrelevant, the function will always return 0.

Explanations

This function is used for the variant with the basic frame. This function extends an internal buffer used for creating the header and the selected columns in an SQL statement.

Example

```
// -----
// Define columns
//          "Column in database |Acronym      |Designation"
// -----
ret = CallBack( "AddColumn", "personalnummer      |PNR      |personnel number" );

or:

ret = CallBack( "AddColumn", "p.personalnummer      pnr|PNR      |personnel number" );
```

Callback function long "SetClauses"

Parameters

SQL fragment with optional clauses:

- optional Where clause (for selecting data records or join conditions)
- optional Group By clause
- optional Order By clause

Return value

The return value is irrelevant, the function will always return 0.

Explanations

This function is used for the variant with the basic frame. It allows for indicating an SQL fragment, which is appended to the SQL select statement.

Example

```
ret = Callback( "SetClauses",  
    " where p.kostenstelle = kst.kostenstelle (+) " ||  
    " and p.firmen_nummer = kst.firma (+) " ||  
    " and p.firmen_nummer like " || BV( get_bapi_val( DLG_DATA, "FIR" ) ) ||  
    " order by 3, 1" );
```

Callback function long "MakeList"

Parameters

None.

Return value

The return value of this function should be returned in the script function main(). It has the same meaning as described above for the script function main().

Explanations

This function is used for the variant with the basic frame and will finally generate the list.

The HYDRA system composes a list header from the indicated acronyms and designations and writes it into the file transferred.

An SQL select statement is composed from the columns, tables and clauses indicated by other callback functions, and the selected data records are written in the list file.

The list consists of a header including the acronyms and designations of the columns. The data is included in the following lines.

PLEASE NOTE: Specifying the identifier "DCD=N|" will have the effect that only the acronyms and no designations are output in the header when the list is created. (available as from hymw 7.2.1.75 June 2005)

Example

```
long main()
{
    ...

    //-----
    // Create list
    //-----
    ret = CallBack( "MakeList", "" );

    return ret;
}
```

Callback function long "WriteLn"

Parameters

Line where the list file is to be output.

Return value

The return value is irrelevant, the function will always return 0.

Explanations

This function writes the line indicated as parameter into the list file.

This function is used for the variant with extended options. It does not make sense to use it in connection with the basic frame.

Example

```
long main()
{
    ...

    ret = CallBack( "WriteLn", "FIR=Firma|PNR=Personalnummer|NAME=Name|BER=Area|KST=Costcenter|" );

    ...

    line = "";
    line = add_bapi_val( line, "", fir );
    line = add_bapi_val( line, "", pnr );
    line = add_bapi_val( line, "", name );
    line = add_bapi_val( line, "", ber );
    line = add_bapi_val( line, "", kst );
    ret = CallBack( "WriteLn", line clipped );

    return ret;
}
```

10.1.1 Troubleshooting

Troubleshooting with extended options

The following errors known from the section above are also handled automatically when the extended options are used:

1. Syntax error in script
2. Runtime error in script
3. The indicated file cannot be opened.

All other errors which may occur during the execution of the function `main()` and the other, independently defined functions activated within it, must be treated there. If an error is detected, processing is to be interrupted and the `main()` function is to be exited with a return value not equal to 0. A debug output with the function `eprint` and/or `pprint` is recommended for improved traceability (please refer to the section "Debug outputs" below).

Logging of SQL or system errors

SQL and system errors are automatically logged in log files in the `err` directory on the HYDRA server. The log file is named "`hymwb.<UserNr>.err`", "`hymw.<UserNr>.err`", "`hybapi.<UserNr>.err`" or "`hyddi.<UserNr>.err`", depending on which program was started (example: `d:\hydra\err\hymwb.1109.err`). In multi-system installations, it is to be noted that the `err` directory is located in the subdirectory featuring the respective system number (example `d:\hydra\2\err\hymwb.1109.err`).

Debug outputs

The functions `pprint` and `eprint` are used for outputs in log files. Please also refer to the description of the HYDRA script language. The function `eprint` writes into the same log files as described in the previous section "Logging of SQL or system errors".

In the debug mode or with activated logging, outputs with the command `dprint` are written in the relevant log files in the error directory of the server and/or on the screen.

10.3 Introduction to the basic frame

The following sections provide an introduction to the creation of PDM lists by using specific examples in which the individual new functions are explained in detail.

10.3.1 Example 1: Simple list of persons

In this example, all persons of the HR master data are to be listed. A restriction to a group of persons is not required.

The basic frame is sufficient for the list definition:

```

1: hydra basic;
2:
3: // -----
4: //
5: // Tutorial for HYDRA-Applications
6: // PDM-List persons
7: //
8: // Variant 1: Simple version
9: //
10: // -----
11:
12: /*-----*/
13: long main()
14: {
15:     ret          long;
16:
17:     // -----
18:     // Define table(s)
19:     // -----
20:     ret = CallBack( "SetTables", "HR master data" );
21:
22:     // -----
23:     // Define columns
24:     // "Column in database |Acronym      |Data type"
25:     // -----
26:     ret = CallBack( "AddColumn", "personalnumber      |PNR      |integer" );
27:     ret = CallBack( "AddColumn", "firmen_nummer      |FIR      |string" );
28:     ret = CallBack( "AddColumn", "person_name      |NAME     |string" );
29:     ret = CallBack( "AddColumn", "bereich         |BER      |string" );
30:     ret = CallBack( "AddColumn", "kostenstelle     |KST      |string" );
31:
32:     // -----
33:     // Define clauses (optional)
34:     // A Where clause, an optional Group-By clause and an optional
35:     // Order-By clause may be defined.
36:     // -----
37:
38:     // Not applicable in this example
39:
40:     // -----
41:     // Create list
42:     // -----
43:     ret = CallBack( "MakeList", "" );
44:
45:     return ret;

```



```
46: }  
47:  
48: /*-----*/  
49:
```

Result:

After the list is created, the file has the following contents:

```
PNR=integer|FIR=string|NAME=string|BER=string|KST=string|  
999998|BSP|Meier|123|105|  
999999|BSP|Schulz|123|105|  
906075|BSP|Erhard|077|105|  
1004|BSP|Hirsch|077|105|  
1009|BSP|Mustermann|077|105|  
400000|BSP|Kron|123|105|
```

Explanation:

The **main()** function is always activated in the created script. In this function, the table "HR master data" is indicated via the callback function "**SetTables**".

Subsequently, the required columns are indicated by activating the callback function "**AddColumn**".

Finally, activation of the callback function "**MakeList**" instructs the HYDRA system to create the defined list and write it into the file.

10.4 Introduction to extended options

10.4.1 Example 2: List of persons

This example is an extension of the previous one. The name and first name are to be separated by a comma, but only if a first name is entered in the HR master data. Otherwise, only the last name is to be indicated.

A special flag for cost center recording is to be set if the cost center includes the key "(ERF)" in its designation.

The list is implemented with the "extended options":

```

1: hydra basic;
2:
3: // -----
4: //
5: // Tutorial
6: // List of people logged on
7: //
8: // Variant 2: Version with extended options
9: //
10: // -----
11:
12: import      DLG_DATA      char(30000);
13:
14: long main()
15: {
16:     ret          long;
17:     line          char(8000);
18:     kst_bez       char(100);
19:     vorname       char(80);
20:     nachname      char(80);
21:     name          char(170);
22:     op_kst_erfass char(10);
23:
24:     ret = Callback( "WriteLn",
25: "FIR=string|PNR=integer|NAME=string|BER=string|KST=string|OP_KST_ERFASS=string|" );
26: /* AAAA */
27: // -----
28: // Declare cursor
29: sqlexec( "declare list_curs cursor for " ||
30: " select p.firmen_nummer, " ||
31: " p.personalnummer, " ||
32: " p.person_name, " ||
33: " p.person_vorname, " ||
34: " p.bereich, " ||
35: " p.kostenstelle, " ||
36: " kst.bezeichnung " ||
37: " from personalstamm p, outer kostenstellen kst " ||
38: " where p.kostenstelle = kst.kostenstelle (+) " ||
39: " and p.firmen_nummer = kst.firma (+) " ||
40: " and p.firmen_nummer like " || BV( get_bapi_val( DLG_DATA, "FIR" ) ) ||
41: " order by 3, 1" );
42: if( sqlcode() = 0 )
43: {
44: /* BBBB */
45: // -----
--

```

```

46:      // Open cursor
47:      sqlexec( "open list_curs;" );
48:      if( sqlcode() != 0 )
49:      {
50:          dprint( "SQL-Fehler" || sqlcode() || " pos" || sqleroffset() );
51:          eprint( "u_l_personen3: Error when opening the cursor" );
52:          ret = 1731;
53:      }
54:
55:      // -----
--
56:      // Loop as long as data records are available
57:      while( sqlcode() = 0 )
58:      {
59:      /* CCCC */
60:          sqlexec( "fetch list_curs;" );
61:          if( sqlcode() = 0 )
62:          {
63:              // Adopt columns to be processed further in internal variables
64:              nachname = SqlColumn( 3 );
65:              vorname  = SqlColumn( 4 );
66:              kst_bez  = SqlColumn( 7 );
67:
68:
69:      /* DDDD */
70:              // Combine name and first name if a first name is entered
71:              if( vorname is not null )
72:              {
73:                  name = nachname clipped || ", " || vorname clipped;
74:              }
75:              else
76:              {
77:                  name = nachname;
78:              }
79:
80:      /* EEEE */
81:              // If the cost center is flagged, set a recording flag
82:              if( pos( "(ERF)", kst_bez ) > 0 )
83:              {
84:                  op_kst_erfass = "J";
85:              }
86:              else
87:              {
88:                  op_kst_erfass = "N";
89:              }
90:
91:      /* FFFF */
92:              // Format and output data record
93:              line = "";
94:              line = add_bapi_val( line, "", SqlColumn(1) );
95:              line = add_bapi_val( line, "", SqlColumn(2) );
96:              line = add_bapi_val( line, "", name );
97:              line = add_bapi_val( line, "", SqlColumn(5) );
98:              line = add_bapi_val( line, "", SqlColumn(6) );
99:              line = add_bapi_val( line, "", op_kst_erfass );
100:              ret = CallBack( "WriteLn", line clipped );
101:          }
102:      }
103:
104:
105:      /* GGGG */
106:      // -----
-----
107:      // Close cursor
108:      sqlexec( "close list_curs;" );
109:      }
110:      else
111:      {
112:          eprint( "u_l_personen3: Error when declaring the cursor" );
113:          ret = 1731;
114:      }
115:
116:      return ret;
117:  }
118:

```

119:

Result:

After the list is created, the file has the following contents {file name}:

```
FIR=string|PNR=integer|NAME=string|BER=string|KST=string|OP_KST_ERFASS= string|
BSP|906075|Erhard, Anton|077|105|N|
BSP|1004|Hirsch, Harry|077|105|N|
BSP|999998|Meier, Hans|123|105|N|
BSP|1009|Mustermann|077|105|J|
BSP|999999|Schulz, Werner|123|105|N|
```

Explanation:

This script has the full control of the data collection and the creation of the list file within the script.

The following sequence is required for data collection:

1. Declaration of an SQL cursor by a select statement (mark /* AAAA */).
2. Opening of the SQL cursor by an open statement (mark /* BBBB */).
3. Reading of the existing data records by a fetch statement in a loop, as long as data records are available. The data records read are output into the list file within the loop. (marks /* CCCC */ to /* FFFF */).
4. Closing of the SQL cursor by a close statement. (mark /* GGGG */).

Two steps are required for creating the list file:

1. Output header with acronyms and data type (before mark /* AAAA */).
2. Output formatted data lines (mark /* FFFF */) within the loop when reading the data records.

The script functions `sqlexec()`, `sqlcode()`, `SqlColumn()` and `add_bapi_val()` are fixed components of the HYDRA script language and described in a separate document.

10.5 Script examples for the HYDRA label design

10.5.1 Example 1: Basic frame script for person, machine

The badge number ("DLG.KNR") and the workplace/machine number ("DLG.MNR") are transferred from the terminal dialog to the script.

Please note:

In order for the workplace/machine number to be transferred correctly even with a numerical HYDRA basic setting of the machine number, the binding variable must be BVMNR instead of BV in this case.

Script "u_l_pnr_mnr":

```

hydra basic;

// -----
// ETD training (PDM list of persons, workplaces / machines)
// Simple version without join to another table
// -----

import  DLG_DATA  char(30000);

/*-----*/
long main()
{
    ret      long;
    // -----
    // Define table(s)
    // -----
    ret = CallBack( "SetTables", "personalstamm p, maschinen m" );
    // -----
    // Define columns
    //          "Column in database |Acronym |Designation"
    // -----
    ret = CallBack( "AddColumn", "p.personalnummer  |PNR   |integer" );
    ret = CallBack( "AddColumn", "p.karten_nummer   |KNR   |string" );
    ret = CallBack( "AddColumn", "p.person_name     |PNAME  |string" );
    ret = CallBack( "AddColumn", "p.person_vorname  |PVORNAME |string" );
    ret = CallBack( "AddColumn", "m.mgruppe       |MGRP   |string" );
    ret = CallBack( "AddColumn", "m.bezeichnung   |MBEZ   |string" );
    ret = CallBack( "AddColumn", "m.user_c_55     |FU:55  |string" );
    // -----
    // Define clauses (conditions) (optional)
    // A Where clause, an optional Group-By clause and an optional
    // Order-By clause may be defined.
    // -----
    ret = CallBack( "SetClauses",
        " where p.karten_nummer = " || BV( get_bapi_val( DLG_DATA, "DLG.KNR" ) ) ||
        " and m.masch_nr = " || BVMNR( get_bapi_val( DLG_DATA, "DLG.MNR" ) ) ||
        " order by 1 " );
    // -----
    // Create list
    // -----
    ret = CallBack( "MakeList", "" );

    return ret;
}
/*-----*/

```